

Clean Code

Code is read far more often than it is written. Clean code respects the reader.

From Structure to Style

Cohesion and coupling shape how the pieces fit. Clean code is about the pieces themselves — how a single class, method, or line reads.

Simplicity

Is it as simple as it can be — no simpler?

Correctness

Does it do the right thing, provably?

Completeness

Does it handle everything it should?

Learning Objectives

- Explain why readability is an economic decision, not a style preference.
- Apply a simple framework: simplicity, correctness, completeness.
- Write intention-revealing names, small functions, and honest error handling.
- Use AI to generate code — and hold it to the same clean-code bar.

You Read Code 10× More Than You Write It

Every hour you save the next reader — often future-you — is paid back many times over. Clean code is how you make the expensive part (reading) cheap.

Dirty code costs

- Slow to understand → slow to change
- Bugs hide in the noise
- Everyone fears touching it

Clean code pays

- Reads like well-written prose
- Intent is obvious; bugs stand out
- Cheap and safe to evolve

A Simple Framework

You could memorise a hundred rules. Instead, judge every piece of code against three questions:



Simplicity

The simplest thing that reads clearly — good names, small units, no duplication.



Correctness

It does the right thing, handles errors honestly, and surprises no one.



Completeness

It covers the edge cases, cleans up its resources, and is truly finished.

SECTION 01

Simplicity

The simplest code that clearly says what it means.

Names Are Documentation

A good name removes the need for a comment. Reveal intent; drop the noise:

Unclear

```
int d; // elapsed days
List<int> l;
if (u.a && u.b) {
    proc(u);
}
```

Intention-revealing

```
int elapsedDays;
List<int> activeUserIds;
if (user.isActive &&
    user.hasConsent) {
    activate(user);
}
```

Same behaviour. The right names turn a puzzle into a sentence.

Small Functions, One Job

A function should do one thing, at one level of abstraction, and be short enough to hold in your head all at once.



Prefer

- Short functions with clear names
- One level of abstraction each
- Few parameters (ideally 0–2)



Avoid

- Long methods that scroll off-screen
- Mixing high- and low-level detail
- Boolean flags that hide two behaviours

Don't Repeat Yourself

Every piece of knowledge should have one home. Duplication means every change must be made in several places — and one will be forgotten.

- Duplicated logic → duplicated bugs and inconsistent fixes.
- Extract the shared idea into one well-named function or type.
- But beware false DRY: code that looks alike yet changes for different reasons.

Comment the 'Why', Not the 'What'

If the code needs a comment to explain what it does, first try to make the code say it:

Java*Good comments explain intent and trade-offs*

```
// BAD: restates the code
```

```
i++; // increment i
```

```
// GOOD: explains a decision the code can't
```

```
// Retry once: the payment gateway drops the first call after a cold start.
```

```
retryOnce(() -> gateway.charge(order));
```

The best comment is a good name. The next best explains a 'why' the code can't.

SECTION 02

Correctness

Clean code that does the wrong thing is just tidy failure.

It Must Do the Right Thing

Readability serves correctness — not the other way round. Clean code makes correctness visible: you can see that it is right.

- Make the expected behaviour obvious from the code's shape.
- Prefer code whose correctness you can reason about locally.
- Tests pin correctness down — we devote a whole session to them soon.

Handle Errors Honestly

Don't hide failure behind magic values or empty catches. Fail fast, fail loud, fail clearly:

Hides failure

```
User find(int id) {  
  try {  
    return db.get(id);  
  } catch (Exception e) {  
    return null; // swallowed  
  }  
}
```

Honest

```
User find(int id) {  
  User u = db.get(id);  
  if (u == null)  
    throw new NotFound(id);  
  return u;  
}
```

A swallowed exception turns a clear failure into a mysterious one later.

The Principle of Least Astonishment

Code should behave the way its name and shape promise. Surprises are where bugs and misuse live.



Astonishing

- `getUser()` also writes to the database
- A 'safe' method throws on valid input
- Names that lie about what they do



Predictable

- Queries read; commands change
- Behaviour matches the name
- No hidden side effects

SECTION 03

Completeness

The 80% that works is not the same as done.

Cover the Edges

Happy-path code is a demo, not a feature. Completeness means the awkward inputs are handled on purpose, not by accident.

- Empty, null, zero, negative, and huge inputs — decide what each does.
- Boundaries: first and last element, off-by-one, overflow.
- Concurrency and ordering, where they apply.
- Make the decision explicit in code — don't leave it to chance.

Clean Up After Yourself

Files, connections, locks — whatever you open, you must close, even when something throws:

Java

```
try (var in =  
    open(path)) {  
    return read(in);  
} // auto-closed
```

C#

```
using var in =  
    Open(path);  
return Read(in);  
// auto-closed
```

Python

```
with open(path)  
    as f:  
    return read(f)  
# auto-closed
```

Language idioms (try-with-resources, using, with) make cleanup automatic — use them. More on this in Secure Coding.

Finished Means Consistent

Signs of 'not done'

- TODOs and commented-out code
- Mixed styles and naming
- Dead code no one calls

Signs of finished

- One consistent style throughout
- No loose ends or leftovers
- Reads as if written by one hand

Clean Code Serves the Two Forces

Clean code isn't a separate goal — it's how cohesion and coupling show up line by line.



Good names & small units

→ higher cohesion: each piece has one clear job.



Honest interfaces, no surprises

→ lower coupling: callers depend on intent, not internals.



Completeness & cleanup

→ changeability: fewer nasty surprises when things evolve.

AI Writes Fast — You Keep It Clean

AI will happily generate working-but-messy code. Your job shifts from typing to judging: does this meet the bar?

- Review AI output for names, size, duplication, and honest errors.
- Ask for the simplest version — then ask it to justify any complexity.
- Encode your standards in the prompt; make 'clean' the default, not a rewrite.

REMEMBER THESE

Key Takeaways

- ✓ Code is read far more than written — optimise for the reader.
- ✓ Judge every piece by three questions: simple, correct, complete.
- ✓ Names are documentation; small functions do one thing.
- ✓ Handle errors honestly and surprise no one.
- ✓ With AI, you are the reviewer who holds the clean-code bar.

What's Next

MODULE 4 · SATURDAY 18 / TUESDAY 21 JULY

SOLID

with Akin Kaldıroğlu

Five principles that turn 'high cohesion, low coupling' into concrete, repeatable design moves.

Hunt the Smells, Then Clean Up

Catalogue what's wrong with the ACME Orders code, then make it simple, correct and complete.

Refactoring course · Smells & Clean Code

Repo github.com/kaldiroglu/refactoring-course-student-materials

IN THIS SESSION, LOOK AT

- ▶ Exercises/Ex02-SmellInventory.md
- ▶ .../util/Util.java , .../service/PricingHelper.java
- ▶ Slides-PDF/Ch04-CodeSmells.pdf

Questions?

Simple. Correct. Complete.